

Computing Methods for Physics 1 – 3 July 2024

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 13:30 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling, clarity in general.

Part 1 – C++

Write a class `Mesh` to represent a set of points in 3D space via their Cartesian coordinates. Using the named constructor idiom and the `vector` class, provide `Mesh` with the constructors:

1. `Mesh::Regular(int N, double L)` to build an $N \times N \times N$ evenly space grid, using N values of x_i evenly spaced between $-L/2$ to $L/2$, N values of y_i evenly spaced between $-L/2$ to $L/2$, and N values of z_i evenly spaced between $-L/2$ to $L/2$;
2. `Mesh::Random(int N, double L)` to build an irregular grid of N points with x_i , y_i and z_i values uniformly extracted between $-L/2$ to $L/2$;
3. `Mesh(int N)` to build a grid with N points by interactively asking the user for their coordinates.

Write a class `Source` with constructor `Source(Mesh grid, std::string field)`:

- if `field` is "scalar", interactively ask the user to provide a value at each point of `grid` (this is intended to represent a scalar source field);
- otherwise, exit with an informative message.

Finally write a class `Voltage` with:

- a constructor `Voltage(Mesh grid, Source charges)` to represent discretely the electrostatic potential V generated by a set of point charges $\{q_i\}$: this will loop over `charges` ($\{q_i, \vec{r}_i\}$) to calculate and store $V_j = V(\vec{r}_j) = \sum_i q_i / |\vec{r}_i - \vec{r}_j|$ at every point in `grid` ($\{\vec{r}_j\}$), where $|\vec{r}_i - \vec{r}_j|$ is the distance between where you are calculating V and the point where the source charge q_i is located;
- a method to print to text file the discrete representation of V sourced by $\{q_i\}$ at $\{x_j, y_j, z_j, V_j\}$.

Part 2 – Python

Implement a class `Solid` to represent rotating solids with uniform density. Provide it with attributes to represent the mass M of the rotating body, three characteristic length scales (some solids will have one or two of these set to zero), its angular rotation frequency ω , its moment of inertia I , its kinetic energy K , and the distance d between the rotation axis and an axis parallel to it that passes through the center of mass (CoM).

Note that calculating I requires knowing the specific shape and mass of the solid; further

$$K = \frac{1}{2}I\omega^2$$

cannot be calculated without knowing I and how the solid spins (i.e., ω and d ; recall that $I = I_{\text{CoM}} + Md^2$).

Also implement three concrete classes that inherit from `Solid`:

1. **Sphere**, to represent a rotating solid sphere with radius R ,
2. **Rod**, to represent a thin solid rod with length L that rotates along an axis perpendicular to it, and
3. **Rectangle**, to represent a thin $L \times W$ rectangular slate that rotates along an axis perpendicular to the plane containing it.

The moments of inertia of these solids, with respect to a rotation axis through their CoMs, are:

$$\begin{aligned}I_{\text{Sphere}} &= \frac{2}{5}MR^2 \\I_{\text{Rod}} &= \frac{1}{12}ML^2 \\I_{\text{Rectangle}} &= \frac{1}{12}M(L^2 + W^2)\end{aligned}$$

Provide each class with getters, setters, a meaningful output for `print`, and with a method `Spin` to set the object in rotation (d and ω are the inputs of this method).